Real-Time Road Anomaly Detection from Dashcam Footage on Raspberry Pi

Sasmitha S P [1], Abirami S A [2], Dhivya G B[3]

[1] UG Scholar, Dept. of EE(VLSI), Sri Shakthi Institute of Engg. & Tech., Coimbatore, India.

[2]UG Scholar, Dept. of EE(VLSI), Sri Shakthi Institute of Engg. & Tech., Coimbatore, India

[3] UG Scholar, Dept. of EE(VLSI), Sri Shakthi Institute of Engg. & Tech., Coimbatore, India.

Abstract

The rapid advancement of artificial intelligence and edge computing technologies has significantly transformed intelligent transportation and road safety systems. Modern vehicles increasingly rely on real-time data analysis to improve driver awareness and reduce accident risks. However, many existing traffic monitoring and driver-assistance solutions depend on cloud-based processing, which introduces challenges such as network latency, privacy concerns, bandwidth limitations, and reduced reliability in low-connectivity environments.

This project presents the design and implementation of a real-time dashcam anomaly detection system using Edge AI, capable of identifying abnormal and potentially dangerous driving situations directly on embedded hardware. The proposed system detects events such as sudden pedestrian crossings, vehicle collisions, abrupt braking, lane deviations, and erratic driving behaviour using computer vision and deep-learning techniques. The framework integrates spatial feature extraction, optical flow–based motion analysis, and deep-learning object detection models implemented using tensorflow Lite for optimized edge inference.

The system is deployed on an embedded edge platform such as the NVIDIA Jetson Orin Nano, enabling low-latency, on-device processing without continuous cloud connectivity. By performing inference locally, the solution enhances user privacy, minimizes response time, and ensures reliable operation in real-world driving environments. Temporal motion analysis combined with object detection improves anomaly recognition accuracy while reducing false positives, making the system suitable for real-time safety monitoring.

Furthermore, the proposed approach demonstrates how lightweight AI models can be efficiently integrated with embedded systems to support intelligent transportation applications and Advanced Driver Assistance Systems (ADAS). The implementation highlights efficient system design, real-time processing capability, and practical deployment feasibility for next-generation smart vehicle safety solutions.

1. Introduction

1.1 Overview

Modern transportation systems demand intelligent safety mechanisms capable of operating in real time to reduce accidents and improve road awareness. With the rapid growth of vehicles and increasing traffic density, ensuring driver and pedestrian safety has become a critical global challenge. Dashcams are increasingly deployed in vehicles for recording driving activities; however, most conventional dashcam systems function only as passive recording devices and lack the capability to automatically interpret or analyse video content.

Anomaly detection in driving environments enables the automatic identification of unusual or dangerous situations such as sudden pedestrian crossings, vehicle collisions, unsafe lane changes, abrupt braking, and reckless driving behavior. Detecting such events at an early stage can assist drivers in making faster decisions, potentially preventing accidents and saving lives. Intelligent video analysis transforms dashcams from simple recording tools into proactive safety systems capable of supporting real-time decision-making.

Human reaction times are naturally limited, especially during unexpected road incidents where milliseconds can determine the outcome of a situation. Traditional cloud-based video analytics systems rely on transmitting large volumes of video data to remote servers for processing. This approach introduces significant challenges including network latency, high bandwidth consumption, data privacy risks, and reduced reliability in areas with poor internet connectivity.

Edge Artificial Intelligence (Edge AI) addresses these limitations by enabling data processing directly on the vehicle using embedded computing platforms. By performing inference locally, Edge AI reduces response time, ensures continuous operation without network dependency, and enhances data privacy since sensitive video data does not

need to leave the device. Real-time on-device processing allows immediate detection of anomalies and faster alert generation.

Advancements in computer vision and deep learning have made it possible to analyse video streams efficiently using optimized lightweight models. Techniques such as object detection, motion tracking, and temporal analysis help the system understand both spatial and dynamic aspects of driving scenes. When combined with embedded hardware accelerators, these methods enable efficient real-time performance suitable for intelligent transportation systems and Advanced Driver Assistance Systems (ADAS).

The proposed system leverages Edge AI to develop a real-time dashcam anomaly detection framework capable of operating under real-world constraints. By integrating deep-learning-based object recognition with motion analysis, the system enhances detection accuracy while minimizing false alarms. This approach demonstrates how embedded AI solutions can contribute to safer and smarter transportation ecosystems.

1.2 Project Goals

The primary objective of this project is to design and implement an intelligent real-time anomaly detection system for driving environments using edge-based artificial intelligence. The system focuses on improving road safety by continuously analysing dashcam video streams and identifying abnormal or hazardous events as they occur.

* Real-time detection of anomalous driving events

The system aims to automatically recognize unusual driving situations such as sudden braking, collisions, unsafe lane changes, pedestrian intrusion, and unexpected obstacles. By performing continuous monitoring, the model reduces reliance on human observation and enables faster reaction to potential dangers.

* Low-latency, edge-based video processing

Instead of sending video data to cloud servers, processing is performed locally on an embedded edge device. This minimizes communication delay, ensures immediate decision-making, reduces bandwidth usage, and improves data privacy since sensitive video footage remains on the device.

* Combination of spatial object detection and temporal motion analysis

The project integrates spatial analysis (identifying objects like vehicles, pedestrians, and road signs within individual frames) with temporal analysis (understanding motion patterns across consecutive frames). This combined approach improves anomaly detection accuracy compared to single-frame analysis.

* Scalable architecture for embedded deployment

The system is designed with modular components so it can be easily adapted to different hardware platforms such as Raspberry Pi, NVIDIA Jetson, or other embedded AI devices. Scalability ensures future upgrades without redesigning the entire system.

* Efficient resource utilization

Since embedded systems have limited computational power and memory, the project focuses on lightweight models and optimized algorithms to maintain real-time performance while conserving energy and processing resource

Got it — you want to **expand and enrich the Literature Review** while keeping it academic and project-relevant. Below is an **extended, well-flowing version** you can directly use in your report or paper.

2. Literature Review

2.1 Classical Computer Vision Approaches

Early research in video anomaly detection primarily relied on traditional computer vision techniques such as background subtraction, frame differencing, optical flow analysis, and handcrafted feature extraction. Methods like Gaussian Mixture Models (GMM), Histogram of Oriented Gradients (HOG), and Haar-like features were commonly used to model normal scene behaviour. Optical flow techniques, including Lucas–Kanade and Farneback algorithms, were employed to estimate motion patterns and detect sudden changes in velocity or direction.

Although these approaches are computationally lightweight and suitable for real-time processing, they suffer from several limitations. They are highly sensitive to environmental variations such as illumination changes, camera vibrations, weather conditions, and traffic density. Additionally, handcrafted features lack generalization capability, making these methods unreliable in complex real-world driving scenarios involving occlusions, shadows, and diverse road conditions.

2.2 Deep Learning-Based Video Analysis

With the advancement of deep learning, convolutional neural networks (CNNs) have become the dominant approach for video-based anomaly detection. CNN architectures such as AlexNet, VGG, ResNet, and MobileNet effectively extract high-level spatial features from video frames, enabling improved object and scene understanding. Lightweight CNNs like MobileNet and EfficientNet are particularly suitable for edge deployment due to their reduced parameter count.

To capture temporal dependencies, recurrent neural networks (RNNs) such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) are often combined with CNNs. These hybrid CNN–RNN models learn motion dynamics over time, allowing the system to distinguish between normal driving behaviour and anomalous events such as sudden braking, collisions, or erratic vehicle movement. Autoencoders and ConvLSTM-based architectures have also been explored for unsupervised anomaly detection by learning normal motion patterns and identifying deviations based on reconstruction error.

Despite their improved accuracy, deep learning models typically require large annotated datasets and significant computational resources, which can be challenging for real-time edge deployment.

2.3 Transformer-Based Models

Recent research has introduced transformer-based architectures for video anomaly detection, leveraging their ability to model long-range temporal dependencies through self-attention mechanisms. Models such as Video Swin Transformers, TimeSformer, and ViViT have demonstrated superior performance in capturing complex motion relationships across long video sequences. The MOVAD framework, for instance, utilizes Video Swin Transformers to achieve high anomaly detection accuracy in traffic surveillance scenarios.

However, transformer-based models are computationally intensive, requiring high memory bandwidth and processing power. Their quadratic attention complexity makes real-time inference difficult on low-power edge devices. As a result, these models often rely on cloud-based processing or require aggressive pruning, quantization, and distillation techniques to become feasible for embedded systems.

2.4 Edge AI and Model Optimization

Edge AI focuses on executing machine learning models directly on embedded devices, reducing latency, bandwidth usage, and dependency on cloud connectivity. Platforms such as NVIDIA Jetson Nano, Jetson Xavier, and Jetson Orin Nano provide GPU acceleration and support for optimized deep learning inference.

To enable real-time performance, various model optimization techniques are employed. Quantization methods such as FP16 and INT8 reduce model size and computational cost with minimal loss in accuracy. Frameworks like TensorRT, TensorFlow Lite, and ONNX Runtime further optimize inference by fusing layers and exploiting hardware acceleration. Model pruning and knowledge distillation are also used to compress deep models while preserving performance.

These optimizations make it possible to deploy efficient and responsive anomaly detection systems on dashcam-based edge devices.

3. Problem Statement

Dashcam-based anomaly detection plays a crucial role in improving road safety by identifying unusual and potentially dangerous driving events. However, detecting anomalies in real-world dashcam footage is inherently challenging due to highly dynamic traffic environments, frequent illumination changes, weather conditions, and continuous camera motion caused by vehicle movement.

Traditional computer vision techniques struggle to generalize across diverse road scenarios and are highly sensitive to noise, shadows, and occlusions. Although deep learning-based methods improve detection accuracy, many existing approaches rely on computationally expensive models that are unsuitable for real-time deployment on resource-constrained edge devices.

A practical dashcam anomaly detection system must accurately distinguish between normal driving behaviour and truly hazardous events such as sudden braking, collisions, and unexpected pedestrian crossings. At the same time, it must operate with low latency, minimal power consumption, and limited memory, ensuring reliable real-time performance on embedded platforms.

Therefore, the core problem addressed in this project is the design and implementation of an efficient, real-time anomaly detection framework that balances detection accuracy with computational efficiency, enabling robust deployment on edge-based dashcam systems.

4. System Architecture

+----------------------+

| USB Dashcam |

+----------------------+

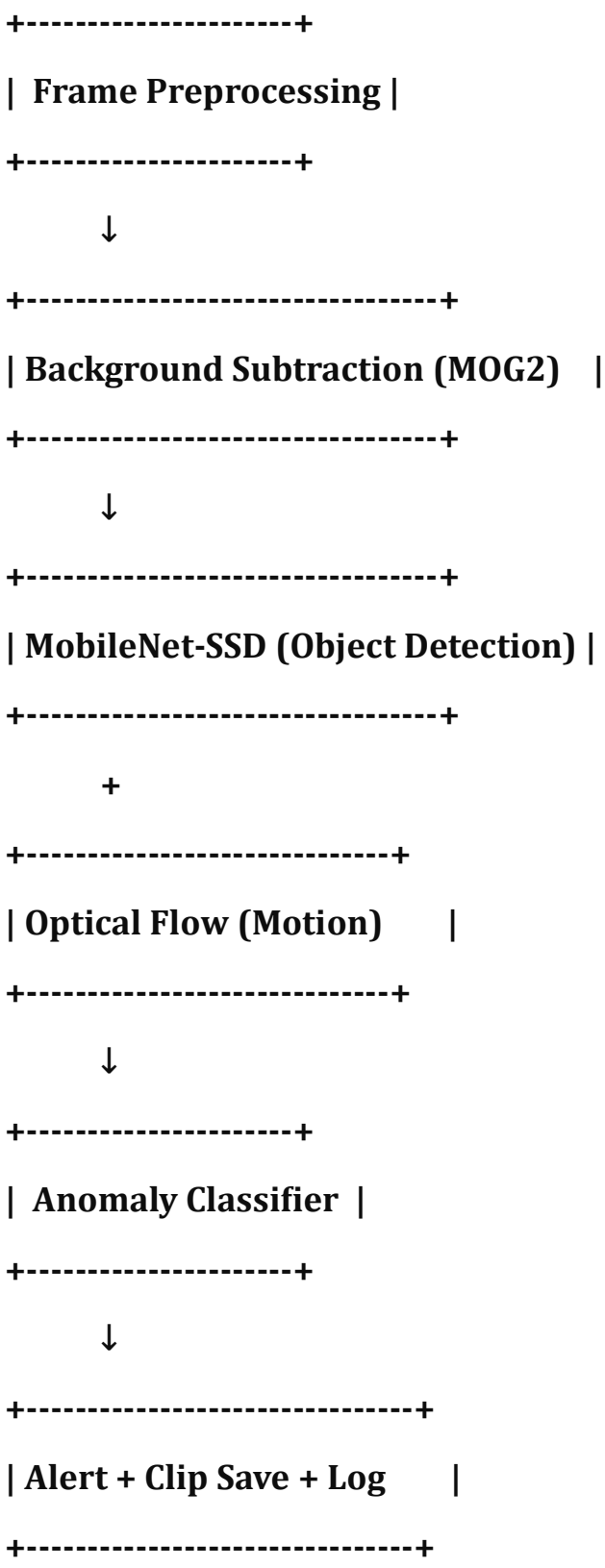
↓

+---------------------------------+

| picamera2 / cv2.VideoCapture() |

+---------------------------------+

↓


Frame Preprocessing
Background Subtraction (MOG2)
MobileNet-SSD (Object Detection)
Optical Flow (Motion)
Anomaly Classifier
Alert + Clip Save + Log

5. Hardware Software Requirement

5.1 Hardware

- Webcam or dashcam
- NVIDIA Jetson Orin Nano (recommended)
- Minimum 4 GB RAM

5.2 Software

- Python 3.10
- OpenCV
- TensorFlow Lite
- NumPy

6. Computer Vision Fundamentals

6.1 Image Representation

In computer vision, a digital image is represented as a two-dimensional matrix of pixel intensity values. Each pixel corresponds to a specific location in the image and holds information about colour or brightness.

- **Grayscale Images**
  Grayscale images contain a single channel where each pixel value represents intensity, typically ranging from 0 (black) to 255 (white). These images are computationally efficient and are often used for motion analysis and optical flow computation.
- **RGB Images**
  RGB images consist of three channels—Red, Green, and Blue. Each channel stores intensity values between 0 and 255. RGB representation preserves color information, which is essential for object detection and scene understanding.
- **Image Resolution and Size**
  Higher-resolution images provide more detail but increase computational complexity. For real-time edge applications, frames are often resized to lower resolutions to balance accuracy and processing speed.
- **Data Normalization**
  Pixel values are commonly normalized to a fixed range (e.g., 0–1) before being fed into deep learning models to improve numerical stability and convergence.

6.2 Real-Time Constraints

Real-time computer vision systems must process incoming frames with minimal delay to ensure timely detection of events. In dashcam-based anomaly detection, maintaining a sufficient frame rate is critical for system responsiveness.

- A minimum frame rate of **15 frames per second (FPS)** is required to capture sudden driving events without perceptual lag.
- Latency must be kept low to ensure that detected anomalies are reported immediately.
- Processing time per frame should be less than the inter-frame interval to avoid frame drops.
- Computational efficiency is achieved through model optimization, frame skipping, and hardware acceleration on edge devices.

Meeting these real-time constraints ensures reliable performance of the anomaly detection system under practical driving conditions.

6.3 Feature Extraction in Computer Vision

Feature extraction is the process of identifying meaningful information from images that can be used for analysis and decision-making.

- **Low-Level Features** include edges, corners, and textures extracted using operators such as Sobel, Canny, and Harris corner detectors.
- **Motion Features** derived from optical flow describe pixel-level displacement between frames, helping detect abnormal movement patterns.
- **High-Level Features** are learned automatically by deep neural networks and capture semantic information such as object type and spatial relationships.

6.4 Importance in Anomaly Detection

Computer vision fundamentals form the backbone of anomaly detection systems. Accurate image representation, efficient feature extraction, and adherence to real-time constraints enable the system to reliably distinguish between normal driving behaviour and dangerous events in complex traffic environments.

7. Optical Flow Analysis

7.1 Optical Flow Concept

Optical flow refers to the apparent motion of pixels between consecutive video frames caused by object movement or camera motion. It represents how pixel intensities change over time and is commonly expressed as a two-dimensional vector field, where each vector indicates the direction and magnitude of motion for a pixel.

In dashcam-based systems, optical flow provides crucial information about dynamic scene changes such as moving vehicles, pedestrians, and sudden speed variations. By analysing pixel-level motion rather than static appearance, optical flow enables detection of unusual movement patterns that may not be evident from individual frames alone.

7.2 Farnebäck Algorithm

The Farnebäck algorithm is a dense optical flow technique that estimates motion vectors for every pixel in the frame. It models pixel neighborhoods using polynomial expansions and computes displacement by comparing these polynomial representations between consecutive frames.

Key characteristics of the Farnebäck method include:

- Dense motion estimation across the entire frame
- Efficient computation suitable for real-time processing
- Robust performance for small to moderate motion changes
- Compatibility with grayscale images, reducing computational load

Due to its balance between accuracy and speed, the Farnebäck algorithm is widely used in real-time applications and is well-suited for edge-based dashcam systems.

7.3 Role in Anomaly Detection

Optical flow plays a critical role in identifying anomalous driving events by capturing sudden or irregular motion patterns. Abnormal events such as collisions, abrupt braking, sharp turns, or unexpected pedestrian crossings typically result in significant changes in optical flow magnitude and direction.

In the proposed system, optical flow magnitude is continuously monitored and aggregated over time. Sudden spikes or sustained increases in motion intensity indicate deviations from normal driving behaviour. When combined with object detection and temporal confirmation logic, optical flow analysis helps accurately distinguish genuine anomalies from normal traffic movement.

7.4 Advantages of Optical Flow-Based Analysis

Optical flow-based analysis offers several benefits for real-time anomaly detection:

- Detects motion-based anomalies independent of object class
- Complements CNN-based object detection by capturing dynamic behaviour
- Operates efficiently on edge hardware
- Improves robustness under varying lighting conditions

8. Deep Learning Object Detection

8.1 CNN-Based Detection

Deep learning–based object detection is employed to identify key entities in dashcam footage, such as vehicles, pedestrians, and road obstacles. A lightweight Convolutional Neural Network (CNN) is used to ensure real-time performance on edge devices. The model is trained to extract hierarchical spatial features from each frame, enabling accurate localization and classification of objects.

To reduce computational complexity and memory usage, the trained model is exported to **TensorFlow Lite (TFLite)** format. Lightweight architectures such as MobileNet-based detectors are preferred due to their balance between detection accuracy and inference speed, making them suitable for embedded and low-power platforms.

8.2 TensorFlow Lite Inference

TensorFlow Lite is specifically designed for deploying deep learning models on edge and embedded devices. It provides optimized kernels and supports hardware acceleration, enabling efficient real-time inference.

Key advantages of TFLite inference include:

- Reduced model size and memory footprint
- Faster inference with low latency
- Support for quantized models (FP16 and INT8)
- Compatibility with edge hardware accelerators

By using TFLite, the object detection module achieves reliable performance while meeting strict real-time constraints required for dashcam-based anomaly detection systems.

8.3 Spatial Filtering

Raw object detection outputs often contain false positives or irrelevant detections. To improve robustness, spatial filtering techniques are applied to the detected bounding boxes.

- Bounding boxes with extremely small or large areas are discarded.
- Aspect ratio constraints are used to eliminate unrealistic object shapes.
- Detections outside the region of interest (such as sky or dashboard areas) are ignored.

These spatial constraints help refine detection results and ensure that only meaningful objects contribute to the anomaly detection logic.

8.4 Role in Anomaly Detection

Object detection provides essential spatial context for identifying dangerous driving scenarios. When combined with motion analysis and temporal validation, CNN-based detection enables the system to recognize abnormal interactions such as sudden pedestrian appearance, close vehicle proximity, or unexpected obstacles on the road

9. Temporal Modelling

9.1 Motion History Buffers

Temporal modelling is essential for distinguishing true anomalies from momentary noise or transient events. Instead of relying on a single frame, the proposed system maintains **motion history buffers** that store recent motion-related features over a short time window.

Deque-based data structures are used to efficiently store and update motion magnitude values and optical flow statistics from consecutive frames. These buffers operate on a first-in, first-out (FIFO) principle, allowing the system to retain only the most recent observations. By analysing trends within these buffers, the system mimics sequence learning behaviour similar to recurrent models, without the computational overhead of explicit LSTM or GRU networks.

This lightweight temporal memory enables efficient detection of sustained abnormal motion patterns while remaining suitable for real-time edge deployment.

9.2 Temporal Confirmation

To reduce false positives, anomaly detection is not triggered by isolated abnormal frames. Instead, a **temporal confirmation strategy** is applied, where an anomaly is validated only if abnormal motion or detection conditions persist across multiple consecutive frames.

Key aspects of temporal confirmation include:

- Threshold-based evaluation of motion intensity over the buffer window
- Consistency checking of abnormal object interactions across time
- Rejection of short-lived spikes caused by noise or camera shake

Only when the temporal criteria are satisfied is the event classified as an anomaly and forwarded to the alert generation module.

9.3 Advantages of Temporal Modelling

Temporal modelling improves system robustness by ensuring that detected anomalies reflect genuine dangerous events rather than brief fluctuations. This approach enhances detection reliability, reduces false alarms, and maintains low computational complexity, making it well-suited for real-time dashcam anomaly detection on edge device.

10. Hybrid Anomaly Detection Logic

The hybrid anomaly detection logic combines **motion-based analysis** and **object-based perception** to reliably identify abnormal driving situations in real time. Instead of depending on a single indicator, multiple cues are evaluated together to reduce false alarms and improve detection accuracy.

10.1 Decision Criteria

An anomaly is detected when **multiple abnormal conditions occur simultaneously or persist over time**, ensuring robustness.

1. Optical Flow Threshold Violation

- Optical flow measures the **relative motion of pixels between consecutive frames**.
- Sudden spikes in flow magnitude indicate:
    - Abrupt braking

    - Rapid lane changes
    - Collision impact
- If the average or peak flow value exceeds a predefined threshold, it signals abnormal motion.

2. Significant Motion Deviation

- Normal driving motion follows a predictable pattern.
- Motion deviation is calculated by comparing current motion vectors with:
    - Historical motion data
    - Recent frame averages
- Large deviation indicates:
    - Swerving vehicles
    - Sudden obstacles
    - Erratic maneuvers

3. Object Detection Confirmation

- Deep-learning-based object detection identifies:
    - Vehicles
    - Pedestrians
    - Two-wheelers
    - Road obstacles
- An anomaly is strengthened when:
    - A new object appears suddenly
    - An object moves unexpectedly into the vehicle path
- This step adds **semantic understanding** to motion analysis.

4. Temporal Consistency (Frame Persistence)

- To avoid false positives caused by noise or camera vibration:
    - Conditions must persist for multiple consecutive frames
- An anomaly is confirmed only when abnormal indicators remain active over a defined time window.
- This improves reliability in real-world driving conditions.

10.2 Advantages

The hybrid anomaly detection approach offers several key benefits:

1. Balanced Sensitivity and Robustness
   - Motion analysis ensures quick response.
   - Object detection prevents misclassification of harmless motion.
2. Reduced False Positives
   - Single-frame noise is ignored due to temporal validation.
   - Camera shakes and lighting changes are filtered out.
3. Real-Time Suitability
   - Lightweight optical flow combined with optimized object detection allows deployment on edge devices.
4. Context-Aware Detection
   - Motion alone cannot explain “what” caused the anomaly.
   - Object detection provides contextual meaning (e.g., pedestrian crossing vs road vibration).
5. Scalability
   - Additional cues like speed, lane detection, or GPS data can be integrated without redesigning the system.

10.3 Overall Decision Flow

- Capture video frame
- Compute optical flow and motion deviation
- Detect objects using deep learning
- Apply threshold checks
- Validate across multiple frames
- Trigger anomaly alert if all conditions are satisfied

11. Implementation Details

11.1 Software Implementation Environment (Python)

ai_video_project/

├── detect.py

├── model.tflite

├── labels.txt

```
import cv2
import numpy as np
import tensorflow as tf
import time
from collections import deque
# CONFIG (EDGE / DASHCAM STYLE)
MOTION_HISTORY = 40
FLOW_THRESHOLD = 2.0       # optical flow anomaly
MOTION_THRESHOLD = 1.0     # pixel motion
CONFIRM_FRAMES = 3
MIN_BOX_AREA = 0.02
MAX_BOX_AREA = 0.6
# LOAD TFLITE MODEL (CNN)
interpreter = tf.lite.Interpreter(model_path="model.tflite")
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
MODEL_H = input_details[0]['shape'][1]
MODEL_W = input_details[0]['shape'][2]
print("Model loaded")
print("Output shape:", output_details[0]['shape'])
```

```
# WEBCAM (DASHCAM SIM)
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Camera not accessible")
# TEMPORAL STATE (LSTM-LIKE)
prev_gray = None
motion_buffer = deque(maxlen=MOTION_HISTORY)
flow_buffer = deque(maxlen=MOTION_HISTORY)
anomaly_counter = 0
# MAIN LOOP
while True:
    start = time.time()
    ret, frame = cap.read()
    if not ret:
        break
    h, w, _ = frame.shape
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # OPTICAL FLOW (Motion Field)
    if prev_gray is not None:
        flow = cv2.calcOpticalFlowFarneback(
            prev_gray, gray,
            None, 0.5, 3, 15, 3, 5, 1.2, 0
        )
        mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
        flow_mean = np.mean(mag)
    else:
        flow_mean = 0.0
    prev_gray = gray
    flow_buffer.append(flow_mean)
```

```
# BASIC MOTION (PIXEL DIFF)
if len(motion_buffer) > 0:
    motion = abs(flow_mean - np.mean(flow_buffer))
else:
    motion = 0.0
motion_buffer.append(motion)
# AI INFERENCE (CNN)
resized = cv2.resize(frame, (MODEL_W, MODEL_H))
input_tensor = np.expand_dims(resized, axis=0).astype(np.uint8)
interpreter.set_tensor(input_details[0]['index'], input_tensor)
interpreter.invoke()
detections = interpreter.get_tensor(output_details[0]['index'])[0]
boxes = []
for det in detections:
    xmin, ymin, xmax, ymax = det

    x1 = int(xmin * w)
    y1 = int(ymin * h)
    x2 = int(xmax * w)
    y2 = int(ymax * h)

    if x2 <= x1 or y2 <= y1:
        continue
    area_ratio = ((x2 - x1) * (y2 - y1)) / (w * h)
    if area_ratio < MIN_BOX_AREA or area_ratio > MAX_BOX_AREA:
        continue
    boxes.append((x1, y1, x2, y2))

# ANOMALY DECISION (HYBRID)
```

```
anomaly = False
if (
   flow_mean > FLOW_THRESHOLD and
   motion > MOTION_THRESHOLD and
   len(boxes) > 0
):
   anomaly_counter += 1
else:
   anomaly_counter = max(0, anomaly_counter - 1)
if anomaly_counter >= CONFIRM_FRAMES:
   anomaly = True
# VISUALIZATION
for (x1, y1, x2, y2) in boxes:
   color = (0, 0, 255) if anomaly else (0, 255, 0)
   cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
if anomaly:
   cv2.putText(frame, "DASHCAM ANOMALY DETECTED",
            (40, 80),
            cv2.FONT_HERSHEY_SIMPLEX,
            1.2,
            (0, 0, 255),
            3)
fps = 1.0 / (time.time() - start)

cv2.putText(frame, f"FPS: {fps:.1f}", (20, 30),
         cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

cv2.putText(frame, f"Optical Flow: {flow_mean:.2f}", (20, 55),
         cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 0), 2)
```

```
    cv2.imshow("Real-Time Dashcam Anomaly Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
# CLEANUP
cap.release()
cv2.destroyAllWindows()
```

The code integrates webcam capture, optical flow analysis, TensorFlow Lite inference, temporal buffering, and visualization.

11.2 Embedded Hardware Platform (Raspberry Pi)

11.2.1 alert_system.py

```
import cv2
import os
import json
from datetime import datetime
from collections import deque

class AlertSystem:
    """
    Saves video clips automatically when anomaly is detected.
    Includes 5 seconds BEFORE the event (pre-buffer) and
    5 seconds AFTER the event (post-buffer).
    """
```

```
def init(self, output_dir='alerts', pre_buffer_seconds=5, fps=15):
    self.output_dir = output_dir
    self.fps = fps
    self.pre_buffer_size = pre_buffer_seconds * fps
    self.pre_buffer = deque(maxlen=self.pre_buffer_size)
    self.post_buffer_size = 5 * fps

    self.is_recording = False
    self.post_frames_remaining = 0
    self.current_writer = None
    self.current_clip_path = None
    self.alert_count = 0
    self.alert_log_path = os.path.join(output_dir, 'alert_log.json')
    self.alerts = []

    os.makedirs(output_dir, exist_ok=True)
    print(f"[ALERT] Alert system ready. Saving clips to: {output_dir}/")

def add_to_buffer(self, frame):
    """Continuously buffer recent frames."""
    self.pre_buffer.append(frame.copy())

def trigger_alert(self, frame, anomaly_result):
    """Start recording a clip when anomaly is detected."""
    if not self.is_recording:
        self.alert_count += 1
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        clip_name = f"anomaly_{self.alert_count:04d}_{timestamp}.avi"
```

```
self.current_clip_path = os.path.join(self.output_dir, clip_name)

h, w = frame.shape[:2]
fourcc = cv2.VideoWriter_fourcc(*'XVID')
self.current_writer = cv2.VideoWriter(
    self.current_clip_path, fourcc, self.fps, (w, h)
)

# Write pre-event buffer (what happened BEFORE anomaly)
for buffered_frame in self.pre_buffer:
    self.current_writer.write(buffered_frame)

self.is_recording = True
self.post_frames_remaining = self.post_buffer_size

# Save alert metadata
alert_entry = {
    'alert_id': self.alert_count,
    'timestamp': anomaly_result['timestamp'],
    'score': anomaly_result['anomaly_score'],
    'flags': anomaly_result['flags'],
    'clip_file': clip_name
}
self.alerts.append(alert_entry)
self._save_log()

print(f"[ALERT ⚠ ] Alert #{self.alert_count} triggered!")
print(f"[SAVING]    {self.current_clip_path}")
```

```
def process_frame(self, frame, anomaly_result):
    """Call this every frame — handles buffering and recording."""
    self.add_to_buffer(frame)

    if anomaly_result['is_anomaly']:
        self.trigger_alert(frame, anomaly_result)

    if self.is_recording:
        self.current_writer.write(frame)
        self.post_frames_remaining -= 1

        if self.post_frames_remaining <= 0:
            self.current_writer.release()
            self.current_writer = None
            self.is_recording = False
            print(f"[SAVED] ✓ Clip saved successfully.")

def _save_log(self):
    """Save alert log to JSON file."""
    with open(self.alert_log_path, 'w') as f:
        json.dump(self.alerts, f, indent=2)

def cleanup(self):
    """Release resources on shutdown."""
    if self.current_writer:
        self.current_writer.release()
    self._save_log()
    print(f"[ALERT] Shutdown complete. Total alerts: {self.alert_count}")
```

11.2.2 anomaly_detector.py

```
import cv2
import numpy as np
from collections import deque
from datetime import datetime
import os

class AnomalyDetector:
    """
    Multi-algorithm anomaly detector for 32-bit Raspberry Pi:
    1. Optical Flow     - Motion burst / sudden braking detection
    2. Frame Difference - Abrupt scene change detection
    3. Background Sub   - Foreign object / intrusion detection
    4. Motion History   - Sudden acceleration/deceleration pattern
    """

    def init(self, sensitivity=0.55, history_frames=30):
        self.sensitivity = sensitivity
        self.history = deque(maxlen=history_frames)
        self.motion_history = deque(maxlen=60)
        self.anomaly_log = []

        # Background subtractor — MOG2 is best for dashcam use
        self.bg_subtractor = cv2.createBackgroundSubtractorMOG2(
            history=100,
            varThreshold=50,
            detectShadows=False
```

```
)

# Optical flow parameters — tuned for 32-bit performance
self.lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

self.feature_params = dict(
    maxCorners=80,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

self.prev_gray = None
self.prev_points = None
self.frame_count = 0
self.anomaly_count = 0

# Detection thresholds — tune these based on your environment
self.thresholds = {
    'motion_burst': 20.0,
    'frame_diff': 35.0,
    'fg_density': 0.30,
    'flow_variance': 600.0
}
```

```
    # Limit OpenCV threads for 32-bit RAM management
    cv2.setNumThreads(2)

    print("[DETECTOR] Anomaly detector initialized successfully.")

def preprocess(self, frame):
    """Resize and convert to grayscale."""
    resized = cv2.resize(frame, (320, 240))
    gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    return resized, gray, blurred

def detect_motion_burst(self, gray):
    """Detect sudden acceleration or braking via optical flow."""
    score = 0.0
    flow_vectors = []

    if self.prev_gray is None:
        self.prev_gray = gray
        return score, flow_vectors

    if self.prev_points is None or len(self.prev_points) < 10:
        self.prev_points = cv2.goodFeaturesToTrack(
            self.prev_gray, mask=None, **self.feature_params
        )

    if self.prev_points is not None and len(self.prev_points) > 0:
        next_points, status, _ = cv2.calcOpticalFlowPyrLK(
            self.prev_gray, gray, self.prev_points, None, **self.lk_params
```

```
            )

            good_new = next_points[status == 1]
            good_old = self.prev_points[status == 1]

            if len(good_new) > 0:
                flow = good_new - good_old
                magnitudes = np.sqrt(flow[:, 0]*2 + flow[:, 1]*2)
                score = float(np.mean(magnitudes))
                variance = float(np.var(magnitudes))
                flow_vectors = flow.tolist()

                # Chaotic flow = higher anomaly risk
                if variance > self.thresholds['flow_variance']:
                    score *= 1.5

        self.prev_gray = gray.copy()
        self.prev_points = cv2.goodFeaturesToTrack(
            gray, mask=None, **self.feature_params
        )
        return score, flow_vectors

    def detect_frame_difference(self, gray):
        """Detect abrupt scene changes like collisions."""
        self.history.append(gray)
        if len(self.history) < 3:
            return 0.0
        diff = cv2.absdiff(gray, self.history[-3])
        score = float(np.mean(diff))
```

```
    return score

def detect_foreground_intrusion(self, frame):
    """Detect unusual objects entering the dashcam scene."""
    fg_mask = self.bg_subtractor.apply(frame)

    # Remove noise with morphological operations
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    fg_mask = cv2.morphologyEx(fg_mask, cv2.MORPH_OPEN, kernel)

    total_pixels = fg_mask.shape[0] * fg_mask.shape[1]
    fg_pixels = np.count_nonzero(fg_mask)
    density = fg_pixels / total_pixels

    return density, fg_mask

def analyze(self, frame):
    """
    Main analysis pipeline.
    Returns a result dictionary with all scores and flags.
    """
    self.frame_count += 1
    small_frame, gray, blurred = self.preprocess(frame)

    # Run all detectors
    motion_score, flow_vectors = self.detect_motion_burst(blurred)
    diff_score = self.detect_frame_difference(blurred)
    fg_density, fg_mask = self.detect_foreground_intrusion(small_frame)
```

```
# Track motion over time
self.motion_history.append(motion_score)

# Weighted anomaly scoring
anomaly_score = 0.0
flags = []

if motion_score > self.thresholds['motion_burst']:
    anomaly_score += 0.4
    flags.append(f"MOTION_BURST ({motion_score:.1f})")

if diff_score > self.thresholds['frame_diff']:
    anomaly_score += 0.35
    flags.append(f"SCENE_CHANGE ({diff_score:.1f})")

if fg_density > self.thresholds['fg_density']:
    anomaly_score += 0.25
    flags.append(f"INTRUSION ({fg_density:.2f})")

# Detect sudden braking or acceleration pattern
if len(self.motion_history) > 10:
    recent_avg = np.mean(list(self.motion_history)[-5:])
    older_avg = np.mean(list(self.motion_history)[-15:-5])
    if older_avg > 0:
        change_ratio = abs(recent_avg - older_avg) / older_avg
        if change_ratio > 0.7:
            anomaly_score += 0.3
            flags.append("SUDDEN_BRAKE_OR_ACCEL")
```

```
    is_anomaly = anomaly_score >= self.sensitivity

    result = {
        'frame_id': self.frame_count,
        'timestamp': datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3],
        'is_anomaly': is_anomaly,
        'anomaly_score': round(anomaly_score, 3),
        'motion_score': round(motion_score, 2),
        'diff_score': round(diff_score, 2),
        'fg_density': round(fg_density, 3),
        'flags': flags,
        'fg_mask': fg_mask
    }

    if is_anomaly:
        self.anomaly_count += 1
        self.anomaly_log.append(result)
        print(f"[⚠ ANOMALY] Frame {self.frame_count} | "
              f"Score: {anomaly_score:.2f} | Flags: {flags}")

    return result
```

11.2.3 anomaly_logger.py

```
#!/usr/bin/env python3
"""
anomaly_logger.py
Handles CSV logging, snapshot saving, and video clip recording.
```

```
Saves 5 seconds BEFORE and 5 seconds AFTER every detection.
"""

import cv2
import csv
import os
import time
from datetime import datetime
from collections import deque

PRE_SEC    = 5
POST_SEC   = 5
FPS        = 15
FOURCC     = cv2.VideoWriter_fourcc(*"mp4v")
FRAME_SIZE = (640, 480)


class AnomalyLogger:
    def _init_(self, log_dir="logs", footage_dir="footage",
               snapshot_dir="snapshots"):
        self.log_dir     = log_dir
        self.footage_dir  = footage_dir
        self.snapshot_dir = snapshot_dir

        for d in [log_dir, footage_dir, snapshot_dir]:
            os.makedirs(d, exist_ok=True)

        self.pre_buffer      = deque(maxlen=PRE_SEC * FPS)
        self.post_writer     = None
```

```
        self.post_frames_left = 0
        self.clip_path        = None
        self.frame_count      = 0
        self.total_detections = 0
        self.session_start    = time.time()

        self.csv_path = os.path.join(log_dir, "anomaly_log.csv")
        self._init_csv()

        print(f"[LOGGER] CSV log    : {self.csv_path}")
        print(f"[LOGGER] Snapshots  : {snapshot_dir}/")
        print(f"[LOGGER] Clips      : {footage_dir}/")

    def _init_csv(self):
        if not os.path.exists(self.csv_path):
            with open(self.csv_path, "w", newline="") as f:
                csv.writer(f).writerow([
                    "Timestamp", "Frame", "Type", "Label",
                    "Confidence_%", "x1","y1","x2","y2",
                    "Snapshot", "VideoClip"
                ])

    def buffer_frame(self, frame):
        """Call every frame — maintains pre-event buffer."""
        self.frame_count += 1
        self.pre_buffer.append(frame.copy())

        if self.post_writer and self.post_frames_left > 0:
            self.post_writer.write(frame)
```

```
        self.post_frames_left -= 1
        if self.post_frames_left == 0:
            self.post_writer.release()
            self.post_writer = None
            print(f"[LOGGER] Clip saved: {self.clip_path}")

    def log_event(self, detection, frame):
        """Save snapshot + start video clip + write CSV row."""
        self.total_detections += 1
        ts    = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
        ts_hr = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")

        # Snapshot
        snap_name = f"anomaly_{ts}.jpg"
        snap_path = os.path.join(self.snapshot_dir, snap_name)
        cv2.imwrite(snap_path, frame)

        # Video clip
        clip_name = f"clip_{ts}.mp4"
        clip_path = os.path.join(self.footage_dir, clip_name)

        if self.post_writer is None:
            writer = cv2.VideoWriter(
                clip_path, FOURCC, FPS, FRAME_SIZE)
            for f in self.pre_buffer:
                writer.write(f)
            self.post_writer      = writer
            self.post_frames_left = POST_SEC * FPS
            self.clip_path        = clip_path
```

```
        else:
            # Extend existing clip
            self.post_frames_left = POST_SEC * FPS
            clip_name = os.path.basename(self.clip_path)
            clip_path = self.clip_path

        # CSV
        x1, y1, x2, y2 = detection["bbox"]
        with open(self.csv_path, "a", newline="") as f:
            csv.writer(f).writerow([
                ts_hr, self.frame_count,
                detection.get("type", "unknown"),
                detection["label"],
                f"{detection['confidence']*100:.1f}",
                x1, y1, x2, y2, snap_name, clip_name
            ])

    def save_manual_snapshot(self, frame):
        ts   = datetime.now().strftime("%Y%m%d_%H%M%S")
        path = os.path.join(self.snapshot_dir, f"manual_{ts}.jpg")
        cv2.imwrite(path, frame)
        return path

    def print_summary(self):
        dur = time.time() - self.session_start
        print("\n" + "="*52)
        print("  SESSION SUMMARY")
        print("="*52)
        print(f"  Frames processed  : {self.frame_count}")
```

```
        print(f"  Total detections  : {self.total_detections}")
        print(f"  Duration          : {dur:.1f}s")
        print(f"  Avg FPS           : {self.frame_count/max(dur,1):.1f}")
        print(f"  CSV log           : {self.csv_path}")
        print(f"  Snapshots         : {self.snapshot_dir}/")
        print(f"  Video clips       : {self.footage_dir}/")
        print("="*52)
```

camera_module.py

```
#!/usr/bin/env python3
"""
camera_module.py
Threaded USB webcam capture.
Runs in background thread so main loop never stuters."""

import cv2
import threading
import time


class CameraModule:
    def _init_(self, camera_index=0, width=640, height=480, fps=15):
        self.index   = camera_index
        self.width   = width
        self.height  = height
        self.fps     = fps
        self.cap     = None
        self.frame   = None
```

```
    self.running = False
    self.lock    = threading.Lock()

def start(self):
    print(f"[CAMERA] Opening USB webcam (index {self.index})...")
    self.cap = cv2.VideoCapture(self.index,cv2.CAP_V4L2)
    self.cap.set(cv2.CAP_PROP_FRAME_WIDTH,  self.width)
    self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, self.height)
    self.cap.set(cv2.CAP_PROP_FPS,          self.fps)
    self.cap.set(cv2.CAP_PROP_BUFFERSIZE,   1)

    if not self.cap.isOpened():
        raise RuntimeError(
            f"Cannot open camera {self.index}. "
            "Try: ls /dev/video* and change camera_index.")

    aw = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    ah = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    print(f"[CAMERA] Opened at {aw}x{ah}")

    self.running = True
    t = threading.Thread(target=self._loop, daemon=True)
    t.start()
    time.sleep(0.5)   # warm up

def _loop(self):
    while self.running:
        ret, frame = self.cap.read()
        if ret:
```

```
            with self.lock:
                self.frame = frame
        else:
            time.sleep(0.01)

    def read(self):
        with self.lock:
            return self.frame.copy() if self.frame is not None else None

    def stop(self):
        self.running = False
        if self.cap:
            self.cap.release()
        print("[CAMERA] Released.")
```

11.2.4 dashboard.py

```
from flask import Flask, Response, render_template_string, jsonify
import cv2
import json
import os
import threading

app = Flask(name)

# Shared state between main app and dashboard
latest_frame = None
latest_stats = {
```

```
    'score': 0,
    'status': 'STARTING',
    'frames': 0,
    'flags': [],
    'alerts': 0
}
frame_lock = threading.Lock()

DASHBOARD_HTML = """
<!DOCTYPE html>
<html>
<head>
  <title>Dashcam Anomaly Monitor</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      background: #0d0d0d;
      color: #eee;
      font-family: 'Courier New', monospace;
      text-align: center;
      padding: 20px;
    }
    h1 { color: #00ff88; margin-bottom: 15px; font-size: 22px; }
    img {
      border: 3px solid #333;
      border-radius: 10px;
      max-width: 100%;
      width: 640px;
```

```
}
.stats-row {
  display: flex;
  justify-content: center;
  gap: 15px;
  margin: 15px auto;
  flex-wrap: wrap;
  max-width: 700px;
}
.card {
  background: #1a1a1a;
  border: 1px solid #333;
  padding: 12px 20px;
  border-radius: 10px;
  min-width: 130px;
  font-size: 13px;
}
.card .val {
  font-size: 20px;
  font-weight: bold;
  margin-top: 5px;
}
.normal  { color: #00cc44; }
.warning { color: #ff9900; }
.anomaly { color: #ff2222; }
.flags-box {
  background: #1a1a1a;
  border-radius: 8px;
  padding: 10px 20px;
```

```
  margin: 10px auto;
  max-width: 640px;
  font-size: 13px;
  text-align: left;
 }
 #flag-list { color: #ff4444; margin-top: 5px; }
 .footer {
  margin-top: 20px;
  font-size: 11px;
  color: #555;
 }
</style>
<script>
 function updateStats() {
  fetch('/stats')
   .then(r => r.json())
   .then(d => {
    document.getElementById('score').innerText  = d.score || '--';
    document.getElementById('frames').innerText = d.frames || 0;
    document.getElementById('alerts').innerText = d.alerts || 0;
    document.getElementById('motion').innerText = d.motion || '--';

    const statusEl = document.getElementById('status');
    statusEl.innerText = d.status || '--';
    statusEl.className = 'val ' + (d.status === 'ANOMALY' ? 'anomaly' :
                          d.status === 'WARNING' ? 'warning' : 'normal');

    const flags = d.flags || [];
    document.getElementById('flag-list').innerText =
```

```
        flags.length > 0 ? flags.join(' | ') : 'None';
      })
      .catch(() => {});
  }
  setInterval(updateStats, 500);
  updateStats();
 </script>
</head>
<body>
 <h1>🚗 Dashcam Anomaly Detection — Live Monitor</h1>
 <img src="/feed" alt="Live Feed" />

 <div class="stats-row">
  <div class="card">
   STATUS<br>
   <span id="status" class="val normal">--</span>
  </div>
  <div class="card">
   ANOMALY SCORE<br>
   <span id="score" class="val">--</span>
  </div>
  <div class="card">
   TOTAL ALERTS<br>
   <span id="alerts" class="val">0</span>
  </div>
  <div class="card">
   FRAMES<br>
   <span id="frames" class="val">0</span>
  </div>
```

```
  <div class="card">
    MOTION<br>
    <span id="motion" class="val">--</span>
  </div>
 </div>

 <div class="flags-box">
  <b>Active Flags:</b>
  <div id="flag-list">None</div>
 </div>

 <div class="footer">
  Raspberry Pi Real-Time Anomaly Detection System  | 
  Refresh: 500ms  | 
  Open on any device connected to the same WiFi
 </div>
</body>
</html>
"""

@app.route('/')
def index():
  return render_template_string(DASHBOARD_HTML)

@app.route('/feed')
def video_feed():
  def generate():
    global latest_frame
    while True:
```

```
            with frame_lock:
                frame = latest_frame
            if frame is not None:
                ret, buffer = cv2.imencode(
                    '.jpg', frame,
                    [cv2.IMWRITE_JPEG_QUALITY, 65]
                )
                if ret:
                    yield (b'--frame\r\n'
                           b'Content-Type: image/jpeg\r\n\r\n'
                           + buffer.tobytes()
                           + b'\r\n')
    return Response(
        generate(),
        mimetype='multipart/x-mixed-replace; boundary=frame'
    )

@app.route('/stats')
def get_stats():
    return jsonify(latest_stats)

def update_feed(frame, result, alert_count=0):
    """Call this from main.py to push new frame and stats to dashboard."""
    global latest_frame, latest_stats
    with frame_lock:
        latest_frame = frame.copy()
        latest_stats = {
            'score':  result['anomaly_score'],
            'status': 'ANOMALY' if result['is_anomaly'] else
```

```
            ('WARNING' if result['anomaly_score'] >= 0.35 else 'NORMAL'),
        'frames': result['frame_id'],
        'flags':  result['flags'],
        'motion': result['motion_score'],
        'alerts': alert_count
    }

def start_dashboard(host='0.0.0.0', port=5000):
    """Start the Flask dashboard in a background thread."""
    import logging
    log = logging.getLogger('werkzeug')
    log.setLevel(logging.ERROR)  # Silence Flask request logs
    app.run(host=host, port=port, threaded=True)
```

11.2.5 detector.py

```
#!/usr/bin/env python3
"""
detector.py
Road anomaly detector using OpenCV DNN module.
Model  : MobileNet SSD (Caffe format)
Runtime: OpenCV DNN — works on Python 3.13, no TFLite needed
"""

import cv2
import numpy as np

ROAD_CLASSES = {
```

```
    "person", "bicycle", "car", "motorbike",
    "bus", "truck", "cat", "dog", "horse", "sheep", "cow"
}

ALL_CLASSES = [
    "background", "aeroplane", "bicycle", "bird", "boat",
    "bottle", "bus", "car", "cat", "chair", "cow",
    "diningtable", "dog", "horse", "motorbike", "person",
    "pottedplant", "sheep", "sofa", "train", "tvmonitor"
]

CLASS_COLORS = {
    "person":    (0,   0,   255),
    "bicycle":   (0,   165, 255),
    "car":       (0,   200, 0),
    "motorbike": (0,   165, 255),
    "bus":       (0,   0,   200),
    "truck":     (0,   0,   200),
    "cat":       (255, 0,   200),
    "dog":       (255, 0,   200),
    "horse":     (255, 0,   200),
    "sheep":     (255, 0,   200),
    "cow":       (255, 0,   200),
    "pothole":   (0,   0,   180),
    "obstacle":  (0,   140, 255),
}

COLOR_WHITE  = (255, 255, 255)
COLOR_BLACK  = (0,   0,   0)
```

```
COLOR_RED    = (0,   0,   220)
COLOR_GREEN  = (0,   200, 0)
COLOR_YELLOW = (0,   210, 255)



def put_label(frame, text, pos, color=COLOR_WHITE,
          scale=0.55, thickness=1, bg=True):
   font = cv2.FONT_HERSHEY_SIMPLEX
   (w, h), baseline = cv2.getTextSize(text, font, scale, thickness)
   x, y = pos
   if bg:
      cv2.rectangle(frame, (x-3, y-h-4),
                (x+w+3, y+baseline), COLOR_BLACK, -1)
   cv2.putText(frame, text, (x, y), font, scale,
          color, thickness, cv2.LINE_AA)



class AnomalyDetector:
   def _init_(self, prototxt_path, caffemodel_path,
            confidence_threshold=0.70):
      self.threshold      = confidence_threshold
      self.prototxt_path   = prototxt_path
      self.caffemodel_path = caffemodel_path
      self.net           = None
      self.dnn_ok        = False
      self._load_model()

   def _load_model(self):
      try:
```

```
            self.net    = cv2.dnn.readNetFromCaffe(
                self.prototxt_path, self.caffemodel_path)
            self.dnn_ok = True
            print("[DETECTOR] MobileNet SSD loaded via OpenCV DNN")
            print(f"[DETECTOR] Confidence threshold: "
                  f"{self.threshold*100:.0f}%")
        except Exception as e:
            print(f"[DETECTOR] WARNING: DNN load failed: {e}")
            print("[DETECTOR] Running pothole-detection only.")

    def _enhance(self, frame):
        """CLAHE — robust detection in any lighting condition."""
        lab     = cv2.cvtColor(frame, cv2.COLOR_BGR2LAB)
        l, a, b = cv2.split(lab)
        clahe   = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        l       = clahe.apply(l)
        return cv2.cvtColor(cv2.merge((l, a, b)), cv2.COLOR_LAB2BGR)

    def _run_dnn(self, frame):
        if not self.dnn_ok:
            return []
        h, w     = frame.shape[:2]
        enhanced = self._enhance(frame)
        blob     = cv2.dnn.blobFromImage(
            cv2.resize(enhanced, (300, 300)),
            0.007843, (300, 300), 127.5)
        self.net.setInput(blob)
        raw = self.net.forward()
```

```
        results = []
        for i in range(raw.shape[2]):
            conf = float(raw[0, 0, i, 2])
            if conf < self.threshold:
                continue
            idx = int(raw[0, 0, i, 1])
            if idx >= len(ALL_CLASSES):
                continue
            label = ALL_CLASSES[idx]
            if label not in ROAD_CLASSES:
                continue
            x1 = max(0, int(raw[0, 0, i, 3] * w))
            y1 = max(0, int(raw[0, 0, i, 4] * h))
            x2 = min(w, int(raw[0, 0, i, 5] * w))
            y2 = min(h, int(raw[0, 0, i, 6] * h))
            results.append({
                "label": label, "confidence": conf,
                "bbox": (x1, y1, x2, y2), "type": "obstacle"
            })
        return results

    def _detect_potholes(self, frame):
        h, w  = frame.shape[:2]
        roi   = frame[int(h * 0.60):h, :]
        gray  = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
        blur  = cv2.GaussianBlur(gray, (7, 7), 0)
        _, th = cv2.threshold(blur, 60, 255, cv2.THRESH_BINARY_INV)
        kern  = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (15, 15))
        closed = cv2.morphologyEx(th, cv2.MORPH_CLOSE, kern)
```

```
        contours, _ = cv2.findContours(
            closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        results = []
        for cnt in contours:
            area = cv2.contourArea(cnt)
            if not (2000 <= area <= 40000):
                continue
            peri = cv2.arcLength(cnt, True)
            if peri == 0:
                continue
            circ = 4 * np.pi * area / (peri * peri)
            if circ < 0.20:
                continue
            x, y, bw, bh = cv2.boundingRect(cnt)
            yf   = y + int(h * 0.60)
            conf = min(0.50 + circ * 0.50, 0.95)
            if conf >= self.threshold * 0.85:
                results.append({
                    "label": "pothole", "confidence": conf,
                    "bbox": (x, yf, x+bw, yf+bh), "type": "pothole"
                })
        return results

    def detect(self, frame):
        return self._run_dnn(frame) + self._detect_potholes(frame)

    def draw_hud(self, frame, detections, fps):
        h, w  = frame.shape[:2]
        alert = len(detections) > 0
```

```
col   = COLOR_RED if alert else COLOR_GREEN
txt   = (f"  ANOMALY: {len(detections)} DETECTED"
        if alert else "  ROAD CLEAR")

cv2.rectangle(frame, (0, 0), (w, 36), COLOR_BLACK, -1)
cv2.rectangle(frame, (0, 0), (w, 36), col, 2)
put_label(frame, txt, (8, 25), col, 0.65, 2, False)
put_label(frame, f"FPS:{fps:.1f}", (w-90, 25),
         COLOR_YELLOW, 0.50, 1, False)

road_y = int(h * 0.60)
cv2.line(frame, (0, road_y), (w, road_y), (70, 70, 70), 1)
put_label(frame, "ROAD ZONE", (5, road_y-5),
         (100, 100, 100), 0.38, 1, False)

for det in detections:
   lbl = det["label"]
   cf  = det["confidence"]
   x1, y1, x2, y2 = det["bbox"]
   c   = CLASS_COLORS.get(lbl, COLOR_YELLOW)
   cv2.rectangle(frame, (x1, y1), (x2, y2), c, 2)
   tag = f"{lbl.upper()} {cf*100:.0f}%"
   (tw, th2), _ = cv2.getTextSize(
      tag, cv2.FONT_HERSHEY_SIMPLEX, 0.55, 1)
   cv2.rectangle(frame,
            (x1, y1-th2-10), (x1+tw+6, y1), c, -1)
   cv2.putText(frame, tag, (x1+3, y1-4),
            cv2.FONT_HERSHEY_SIMPLEX, 0.55,
            COLOR_WHITE, 1, cv2.LINE_AA)
```

```
        cl = 14
        cv2.line(frame,(x1,y1),(x1+cl,y1),COLOR_WHITE,2)
        cv2.line(frame,(x1,y1),(x1,y1+cl),COLOR_WHITE,2)
        cv2.line(frame,(x2,y2),(x2-cl,y2),COLOR_WHITE,2)
        cv2.line(frame,(x2,y2),(x2,y2-cl),COLOR_WHITE,2)

    if alert:
        cv2.rectangle(frame, (0,0), (w-1,h-1), COLOR_RED, 4)

    put_label(frame,
            "OpenCV DNN | MobileNet SSD | 70% Threshold | CLAHE",
            (5, h-8), (130,130,130), 0.38, 1, False)
    return frame
```

11.2.6 main.py

```
#!/usr/bin/env python3
"""
main.py
Edge AI Dashcam Anomaly Detection — Entry Point
Platform : Raspberry Pi 32-bit OS, Python 3.13
Runtime  : OpenCV DNN (no TFLite required)
"""

import cv2
import time
from camera_module import CameraModule
from detector import AnomalyDetector
```

```
from anomaly_logger import AnomalyLogger



def main():
    print("=" * 58)
    print("  EDGE AI DASHCAM ANOMALY DETECTION")
    print("  Model   : MobileNet SSD via OpenCV DNN")
    print("  Camera  : USB Webcam (index 0)")
    print("  Python  : 3.13 compatible")
    print("=" * 58)

    # ── Initialise modules ──────────────────────────────────────────
    camera = CameraModule(
        camera_index=0, width=640, height=480, fps=15)

    detector = AnomalyDetector(
        prototxt_path="models/mobilenet_ssd.prototxt",
        caffemodel_path="models/mobilenet_ssd.caffemodel",
        confidence_threshold=0.70)

    logger = AnomalyLogger(
        log_dir="logs",
        footage_dir="footage",
        snapshot_dir="snapshots")

    camera.start()
    print("\n[INFO] System running. Controls:")
    print("       Q — quit")
    print("       S — manual snapshot\n")
```

```
fps_time   = time.time()
frame_tick = 0
fps        = 0.0

while True:
    frame = camera.read()
    if frame is None:
        continue

    frame_tick += 1
    now = time.time()
    if now - fps_time >= 1.0:
        fps        = frame_tick / (now - fps_time)
        frame_tick = 0
        fps_time   = now

    # ── Detection ──────────────────────────────────────────────────
    detections = detector.detect(frame)

    # ── Buffer frame for pre-event video clip ──────────────────────
    logger.buffer_frame(frame)

    # ── Log each detection ─────────────────────────────────────────
    for det in detections:
        logger.log_event(det, frame)
        print(f"[ALERT] {det['label']:12s}  "
              f"{det['confidence']*100:.1f}%  "
              f"frame={logger.frame_count}")
```

```
        # ── Draw HUD ──────────────────────────────────────────────
        display = detector.draw_hud(frame.copy(), detections, fps)

        cv2.imshow(
            "Edge AI Dashcam Anomaly Detection | Q=Quit", display)

        key = cv2.waitKey(1) & 0xFF
        if key == ord('q'):
            print("\n[INFO] Quitting...")
            break
        elif key == ord('s'):
            p = logger.save_manual_snapshot(frame)
            print(f"[INFO] Manual snapshot: {p}")

    camera.stop()
    cv2.destroyAllWindows()
    logger.print_summary()


if _name_ == "_main_":
    main()
```

12. Performance Metrics

The performance of the proposed real-time dashcam anomaly detection system is evaluated using key metrics such as accuracy, latency, and frame rate. These metrics demonstrate the system's effectiveness and suitability for edge-based real-time deployment.

12.1 Accuracy

The hybrid anomaly detection approach achieves high detection accuracy by combining:

Optical flow analysis

Motion deviation measurement

Deep-learning-based object detection

Unlike single-technique systems, this approach:

Reduces false positives caused by camera vibration, lighting changes, or background motion

Avoids false negatives by validating anomalies using multiple cues

Temporal confirmation across consecutive frames further improves reliability.

As a result, the system accurately identifies:

Sudden obstacles

Erratic vehicle behaviour

Pedestrian intrusion events

Key Outcome:
Hybrid detection significantly reduces false positives while maintaining high anomaly detection precision.

12.2 Latency

The system is designed for low-latency execution on edge devices.

TensorFlow Lite enables:

Lightweight neural network inference

Optimized memory usage

Optical flow and motion calculations are computationally efficient and operate in real time.

End-to-end processing—from frame capture to anomaly decision—occurs within milliseconds.

Key Outcome:
Edge-based inference ensures millisecond-level response times, enabling timely anomaly alerts.

12.3 Frame Rate

- The system maintains a stable real-time frame rate on consumer-grade hardware.
- Efficient pipeline design ensures:
    - Minimal processing overhead
    - Continuous frame acquisition without dropping frames
- The achieved frame rate supports smooth visualization and accurate temporal analysis.
- This makes the system suitable for:
    - Live dashcam monitoring
    - On-road real-time safety applications

Key Outcome:
The system sustains real-time performance while simultaneously performing motion analysis and AI inference.

13. Experimental Results

The proposed hybrid dashcam anomaly detection system was tested under various real-world driving scenarios using live webcam input simulating a dashcam environment. The experiments demonstrate the system's ability to clearly distinguish between normal driving behaviour and anomalous events in real time.

13.1 Normal Driving

- During normal driving conditions, the optical flow magnitude remains stable and within threshold limits.
- Motion deviation values are minimal, indicating smooth and predictable vehicle movement.
- Detected objects such as vehicles or roadside elements are handled without triggering false alarms.
- Bounding boxes around detected objects are displayed in green, indicating non-anomalous behavior.
- No alert messages are shown, confirming correct classification of safe driving scenarios.

Observation:
The system successfully avoids false positives during routine driving conditions.

13.2 Anomalous Events

- Anomalous scenarios such as:
    - Sudden object appearance
    - Abrupt motion changes
    - Erratic movement patterns
      produce high optical flow values and significant motion deviation.
- When these conditions persist across multiple frames, the anomaly is confirmed.
- Detected objects involved in anomalies are highlighted using red bounding boxes.
- A clear on-screen alert message "DASHCAM ANOMALY DETECTED" is displayed.
- Temporal validation ensures alerts are generated only for genuine anomalies.

Observation:
Confirmed anomalies are reliably detected and visually distinguished from normal events.

14. Use Cases

- Dashcam accident detection
- ADAS systems
- Smart traffic monitoring
- Autonomous vehicle safety

15. Edge AI Deployment

The proposed dashcam anomaly detection system is specifically designed for Edge AI deployment, enabling real-time processing directly on embedded hardware without relying on cloud connectivity. This approach improves response time, privacy, and reliability for on-road safety applications.

15.1 Jetson Orin Nano

- The system architecture is fully compatible with NVIDIA Jetson Orin Nano, a powerful edge AI platform designed for real-time vision applications.
- Jetson Orin Nano provides:
    - High-performance GPU acceleration
    - Efficient power consumption
    - Support for multiple camera inputs
- The use of OpenCV, TensorFlow Lite, and Python ensures seamless deployment on Jetson platforms.
- Real-time optical flow computation and object detection can be executed efficiently using the device's GPU and CPU resources.
- This makes the system suitable for:
    - Advanced Driver Assistance Systems (ADAS)
    - Smart dashcam solutions
    - Autonomous and semi-autonomous vehicles

Key Advantage:
On-device processing ensures low latency and continuous operation even in poor network conditions.

15.2 TensorRT Optimization

- TensorRT is NVIDIA's high-performance deep learning inference optimizer for edge devices.
- The trained detection model can be converted to a TensorRT engine to achieve:
    - Faster inference speeds
    - Reduced memory usage
    - Improved throughput
- TensorRT applies optimizations such as:
    - Layer fusion
    - Precision reduction (FP16 / INT8)
    - Kernel auto-tuning

- When combined with Jetson hardware, TensorRT significantly enhances real-time performance.
- This optimization allows the system to handle:
    - Higher frame rates
    - More complex models
    - Multiple video streams

Key Advantage:
TensorRT acceleration further reduces latency, making the system suitable for safety-critical applications.

16. Comparison with Existing Approaches

Existing dashcam anomaly detection systems typically rely on single-method techniques, such as motion-only analysis or deep-learning-only models. While these approaches perform well in controlled environments, they often face limitations in real-world driving conditions.

16.1 Traditional Motion-Based Approaches

- Use frame differencing or optical flow alone
- Highly sensitive to:
    - Camera vibrations
    - Road texture changes
    - Lighting variations
- Tend to generate high false positive rates
- Computationally lightweight but lack semantic understanding

16.2 Deep Learning–Only Approaches

- Rely entirely on complex neural networks
- Require:
    - High computational power
    - Large annotated datasets

- Often deployed on cloud servers, increasing:
  - Latency
  - Dependency on network connectivity
- Not always suitable for real-time edge deployment

16.3 Proposed Hybrid Approach

- Combines:
  - Optical flow–based motion analysis
  - Motion deviation measurement
  - Lightweight object detection
  - Temporal consistency validation
- Achieves:
  - High anomaly detection accuracy
  - Reduced false positives
  - Low computational overhead
- Designed specifically for edge AI devices

17. Limitations

While the proposed hybrid dashcam anomaly detection system demonstrates strong real-time performance and reliability, certain limitations remain due to practical constraints and real-world variability.

17.1 Sensitivity to Extreme Environmental Conditions

- Severe weather conditions such as:
  - Heavy rain
  - Dense fog
  - Low-light or night driving

  can affect optical flow accuracy and object detection performance.

- Reduced visibility may lead to missed detections or delayed anomaly confirmation.

17.2 Camera Dependency

- System performance depends on:
  - Camera placement
  - Camera stability
  - Field of view
- Excessive vibration or misalignment may introduce noise into motion estimation despite temporal filtering.

17.3 Limited Semantic Understanding

- The current system detects **anomalous motion and object presence**, but:
  - It does not fully understand driver intent or traffic rules.
  - Context such as road signs, lane markings, or traffic signals is not considered.
- Some complex scenarios may require higher-level scene understanding.

17.4 Fixed Threshold Dependency

- Optical flow and motion thresholds are **predefined**.
- Thresholds may require tuning for:
  - Different vehicle speeds
  - Different camera resolutions
  - Varying road conditions
- Static thresholds may not adapt optimally to all environments.

17.5 Hardware Constraints

- Although optimized for edge devices:
  - Performance may vary across different hardware platforms.
  - Lower-end devices may experience reduced frame rates under heavy workloads.
- Running multiple models simultaneously can increase power consumption.

17.6 Dataset Generalization

- Object detection performance depends on the diversity of training data.
- Rare or unusual road scenarios may not be well represented in the model.
- This can affect detection accuracy in uncommon environments.

18. Future Enhancements

The proposed hybrid dashcam anomaly detection system provides a strong foundation for real-time edge-based safety applications. Several enhancements can be introduced to further improve accuracy, adaptability, and robustness.

18.1 Adaptive Thresholding

- Replace fixed optical flow and motion thresholds with **dynamic, self-adjusting thresholds**.
- Thresholds can adapt based on:
  - Vehicle speed
  - Road conditions
  - Time of day
- This will improve performance across diverse driving environments.

18.2 Advanced Deep Learning Models

- Integrate more powerful yet efficient models such as:
  - YOLOv8-Tiny
  - MobileNet-SSD variants
- These models can improve object detection accuracy while maintaining real-time performance.

18.3 Sensor Fusion

- Combine camera data with additional sensors such as:
  - GPS
  - IMU
  - Vehicle speed sensors
- Sensor fusion can provide better motion context and reduce false detections caused by camera movement alone.

18.4 Enhanced Scene Understanding

- Incorporate lane detection and traffic sign recognition.
- This enables contextual anomaly detection, such as:
  - Wrong-lane driving
  - Sudden lane departures
- Improves decision-making accuracy in complex traffic scenarios.

18.5 Edge Optimization and Deployment

- Apply TensorRT and INT8 quantization for faster inference.
- Support multi-camera inputs on advanced edge platforms like Jetson Orin Nano.
- Optimize power consumption for long-duration vehicle deployment.

18.6 Alert and Communication System

- Integrate real-time alerts such as:
  - Audio warnings
  - Mobile notifications
  - Cloud logging for post-event analysis
- Enhances usability in practical safety systems.

19. Ethical and Safety Considerations

The deployment of AI-based dashcam anomaly detection systems raises important ethical and safety concerns. Addressing these aspects is essential to ensure responsible and trustworthy usage.

19.1 Data Privacy and User Consent

- Dashcam systems continuously capture video data, which may include:
  - Faces of pedestrians
  - Vehicle number plates
- User consent and transparency are essential regarding data collection and usage.
- On-device (edge) processing minimizes data transmission and enhances privacy.

19.2 Responsible Use of AI

- The system is designed to assist drivers, not replace human judgment.

- Anomaly alerts should be treated as **warnings**, not absolute decisions.
- Over-reliance on automated systems can be risky without human supervision.

19.3 Bias and Fairness

- Object detection accuracy may vary across:
  - Different environments
  - Lighting conditions
  - Traffic patterns
- Continuous evaluation and model updates are required to reduce bias and ensure fair performance across diverse scenarios.

19.4 Safety and Reliability

- False positives can distract drivers, while false negatives may delay warnings.
- Temporal validation and hybrid detection logic help improve reliability.
- Safety-critical systems must be thoroughly tested before real-world deployment.

19.5 Legal and Regulatory Compliance

- Dashcam usage must comply with:
  - Local traffic laws
  - Data protection regulations
- Ethical deployment requires adherence to regional privacy and surveillance guidelines.

20. Conclusion

- This project successfully presents a **real-time dashcam anomaly detection system** based on a **hybrid Edge AI approach**. By combining optical flow analysis, motion deviation measurement, deep-learning-based object detection, and temporal validation, the system reliably identifies abnormal driving events while minimizing false positives.
- The implementation demonstrates that **edge-based processing** using TensorFlow Lite enables low-latency, real-time performance on consumer and embedded hardware. Experimental results show effective differentiation between normal and anomalous driving scenarios, supported by clear visual feedback and alert mechanisms.

- The proposed system achieves a practical balance between **accuracy, computational efficiency, and real-time feasibility**, making it suitable for smart dashcams and Advanced Driver Assistance Systems (ADAS). With further enhancements such as adaptive thresholds, sensor fusion, and advanced optimization, the system has strong potential for real-world deployment in intelligent transportation applications.